مینی پروژه درس برنامه نویسی پیشرفته
مستندات پروژه Foodock
دکتر ثبوتی
بهار ۱۴۰۵

1 .مقدمه

این پروژه یک سامانه سفارش غذای مبتنی بر زبان C++ است که با هدف شبیه‌سازی فرآیندهای اصلی یک پلتفرم واقعی سفارش آنلاین غذا طراحی شده است. سامانه از چند نقش کاربری پشتیبانی می‌کند، اطلاعات را در پایگاه‌داده ذخیره می‌کند، امکان مدیریت رستوران و منو را فراهم می‌سازد و چرخه کامل ثبت و پیگیری سفارش را پوشش می‌دهد. در فاز دوم نیز سیستم با افزودن قابلیت‌هایی مانند امتیاز وفاداری، سطح عضویت، کوپن تخفیف و تاریخچه تغییر سطح توسعه یافته است.

نقطه شروع برنامه در فایل `main.cpp:96` قرار دارد. در این بخش، اتصال پایگاه‌داده مقداردهی می‌شود، DAOها ساخته می‌شوند و برنامه وارد منوی اصلی می‌شود. سپس کاربر می‌تواند ثبت‌نام کند، وارد سامانه شود یا از برنامه خارج شود `main.cpp:109` .

2 .هدف و دامنه پروژه

هدف اصلی پروژه، طراحی یک سامانه کامل برای مدیریت کاربران، رستوران‌ها، آیتم‌های منو و سفارش‌هاست؛ به‌طوری‌که منطق داده، منطق تجاری و رابط کاربری از یکدیگر جدا باشند. در فاز اول، تمرکز اصلی بر زیرساخت سامانه، نقش‌های کاربری، ثبت‌نام، ورود، مدیریت رستوران، مدیریت منو و ثبت سفارش است. در فاز دوم، همین زیرساخت با قابلیت‌های هوشمندتر توسعه داده شده است؛ از جمله سطح‌بندی مشتریان، محاسبه امتیاز، اعمال تخفیف عضویت، مدیریت کوپن، ثبت تاریخچه سطح عضویت و ارائه قابلیت‌های مدیریتی بیشتر برای ادمین .

بنابراین پروژه یک سامانه چندلایه است که ابتدا عملیات پایه سفارش غذا را پیاده‌سازی کرده و سپس در فاز دوم به سمت شخصی‌سازی رفتار مشتری و توسعه منطق تجاری حرکت کرده است.

3 .معماری کلی برنامه

معماری کلی پروژه را می‌توان به سه لایه اصلی تقسیم کرد:

-لایه رابط کاربر و کنترل جریان برنامه

-لایه مدل‌های شی‌گرا

-لایه دسترسی به داده یاDAO

لایه اول عمدتاً در `main.cpp:417` و `main.cpp:915` و `main.cpp:1405` قرار گرفته است. این بخش مسئول نمایش منوها، دریافت ورودی، کنترل جریان برنامه و هدایت کاربر به پنل مناسب است.

لایه دوم شامل کلاس‌های مدل مانند `User` ، `Customer` ، `Restaurant` ، `MenuItem` ، `Order` ، `OrderItem` و `Coupon` است. این کلاس‌ها ساختار داده‌های اصلی سامانه را نگه می‌دارند و موجودیت‌های دامنه مسئله را مدل می‌کنند `Customer.h:10` `User.h:7` .
`Restaurant.h:6` `MenuItem.h:6` `Order.h:6` `OrderItem.h:4` `Coupon.h:8`

لایه سوم شامل DAO هاست. این لایه مسئول جدا کردن کوئری‌های پایگاه‌داده از منطق برنامه است و باعث می‌شود SQL مستقیماً در منوها و کدهای رابط کاربر نوشته نشود `UserDAO.h:9` .
`RestaurantDAO.h:7` `MenuItemDAO.h:8` `OrderDAO.h:1`
`OrderItemDAO.h:1` `CouponDAO.h:10`

این تفکیک معماری یک مزیت مهم دارد: هر بخش از سامانه مسئولیت مشخصی دارد و در نتیجه توسعه، رفع خطا و افزودن قابلیت‌های جدید ساده‌تر می‌شود.

4 .فاز اول: زیرساخت اصلی سامانه

4-1 .راه‌اندازی برنامه و منوی اصلی

در فاز اول، هسته اصلی پروژه شکل گرفته است. برنامه در `main.cpp:96` با راه‌اندازی دیتابیس و ایجادDAO ها آغاز می‌شود. سپس منوی اصلی در `main.cpp:109` نمایش داده می‌شود و کاربر می‌تواند وارد سامانه شود یا در نقش مدیر رستوران و مشتری ثبت‌نام کند. در پایان نیز نمونه Singletonپایگاه‌داده آزاد می‌شود `main.cpp:134` .

این ساختار نشان می‌دهد که از همان ابتدا برنامه بر پایه یک چرخه اجرای منظم طراحی شده و بین وظیفه راه‌اندازی، تعامل با کاربر و آزادسازی منابع تفکیک وجود دارد.

4-2 .نقش‌های کاربری

فاز اول سه نقش اصلی سامانه را تعریف می‌کند:

-مدیر سیستم

-مدیر رستوران

-مشتری

توابع پنل هر نقش در `main.cpp:66` و `main.cpp:81` و `main.cpp:89` اعلان شده‌اند و پیاده‌سازی آن‌ها به‌ترتیب در `main.cpp:417` و `main.cpp:915` و `main.cpp:1405` قرار دارد.

در سطح مدل شی‌گرا، `RestaurantManager` و `SystemAdmin` به‌صورت زیرکلاس‌هایی از `User` پیاده‌سازی شده‌اند `SystemAdmin.h:7` `RestaurantManager.h:7` .این طراحی باعث می‌شود رفتار هر نقش در عین اشتراک در اطلاعات پایه کاربر، بتواند سفارشی‌سازی شود. برای مثال متد `displayMenu` در این کلاس‌ها override شده است `RestaurantManager.h:30` . `SystemAdmin.h:12`

4-3 .مدیریت ورودی و اعتبارسنجی

یکی از نکات مهم در پروژه، توجه به اعتبارسنجی ورودی‌هاست. در فاز اول، مجموعه‌ای از توابع کمکی برای خواندن داده‌های مختلف تعریف شده‌اند. این توابع از ورودی نامعتبر جلوگیری می‌کنند و با تکرار پیام، داده صحیح را از کاربر می‌گیرند `main.cpp:154` `main.cpp:138` . `main.cpp:232` `main.cpp:219` `main.cpp:206` `main.cpp:185`

این طراحی چند مزیت دارد:

-جلوگیری از تکرار کد اعتبارسنجی در بخش‌های مختلف

-کاهش احتمال کرش برنامه به دلیل ورودی نادرست

-بهبود تجربه کاربر در محیط کنسولی

-ساده‌تر شدن منطق پنل‌ها

به‌ویژه وجود تابع دریافت رشته غیرخالی و توابع خواندن عدد غیرمنفی به این دلیل است که برنامه فقط به اجرای دستورات اکتفا نکرده و برای کنترل خطاهای متداول کاربر نیز طراحی شده است . `main.cpp:219` `main.cpp:185`

4-4 .مدل‌های اصلی فاز اول

فاز اول بر پایه چند کلاس اصلی بنا شده است:

`User` -برای نمایش کاربر پایه `User.h:7` .

`Restaurant` -برای نگهداری اطلاعات رستوران `Restaurant.h:6` .

`MenuItem` -برای آیتم‌های منو `MenuItem.h:6` .

`Order` -برای اطلاعات سفارش `Order.h:6` .

`OrderItem` -برای آیتم‌های هر سفارش `OrderItem.h:4` .

کلاس `User` دارای destructor مجازی است که برای حذف صحیح اشیای مشتق‌شده اهمیت دارد `User.h:11` .

کلاس `Restaurant` علاوه بر اطلاعات پایه، وضعیت رستوران، زمان آماده‌سازی و شناسه مدیر را نیز نگه می‌دارد `Restaurant.h:11` .

کلاس `Order` ارتباط سفارش با مشتری و رستوران را حفظ می‌کند و قیمت نهایی و وضعیت سفارش را نیز نگه می‌دارد `Order.h:9` .کلاس `OrderItem` نیز ساختار چندآیتمی بودن سفارش را پشتیبانی می‌کند `OrderItem.h:7` .

4-5 .پایگاه‌داده و مدیریت آن

کلاس `DatabaseManager` اتصال به SQLite را مدیریت می‌کند و از الگوی Singleton استفاده می‌کند تا کل برنامه از یک اتصال مرکزی بهره ببرد `DatabaseManager.h:10` .
`DatabaseManager.cpp:28`

در سازنده این کلاس، دیتابیس باز می‌شود و جدول‌ها ساخته می‌شوند .
`DatabaseManager.cpp:9` در مخرب کلاس نیز اتصال بسته می‌شود .
`DatabaseManager.cpp:21`

در فاز اول، جدول‌های اصلی زیر ایجاد شده‌اند:

`Users` -برای کاربران `DatabaseManager.cpp:51` .

`Restaurants` -برای رستوران‌ها `DatabaseManager.cpp:62` .

`MenuItems` -برای آیتم‌های منو `DatabaseManager.cpp:74` .

`Orders` -برای سفارش‌ها `DatabaseManager.cpp:87` .

`OrderItems` -برای اقلام سفارش `DatabaseManager.cpp:99` .

سفارش و اقلام سفارش در دو جدول جدا نگهداری می‌شوند تا هر سفارش بتواند شامل چندین آیتم باشد. همچنین کلیدهای خارجی بین جداول، یکپارچگی داده‌ها را حفظ می‌کنند .
`DatabaseManager.cpp:83` `DatabaseManager.cpp:70`
`DatabaseManager.cpp:106` `DatabaseManager.cpp:95`

4-6 .لایه DAO در فاز اول

در فاز اول، DAOها نقش اصلی در سازمان‌دهی منطق پایگاه‌داده دارند. این الگو کمک می‌کند عملیات CRUD و کوئری‌های خاص در کلاس‌های اختصاصی نوشته شوند، نه در منوهای برنامه

`UserDAO` -برای کاربران `UserDAO.h:9` .

`RestaurantDAO` -برای رستوران‌ها `RestaurantDAO.h:7` .

`MenuItemDAO` -برای منو `MenuItemDAO.h:8` .

`OrderDAO` -برای سفارش‌ها `OrderDAO.h:1` .

`OrderItemDAO` -برای آیتم‌های سفارش `OrderItemDAO.h:1` .

این لایه‌بندی باعث می‌شود اگر در آینده نوع پایگاه‌داده تغییر کند یا لازم باشد منطق جست‌وجو و بازیابی داده‌ها اصلاح شود، تغییرات فقط در DAO ها انجام شود و بقیه برنامه کمتر تحت تأثیر قرار گیرد.

4-7 .مدیریت سفارش در فاز اول

در این پروژه، `Order` اطلاعات کلی سفارش را نگه می‌دارد و `OrderItem` برای اقلام آن استفاده می‌شود `OrderItem.h:7` `Order.h:9` .منطق مرتبط در `OrderDAO.cpp:1` و `OrderItemDAO.cpp:1` پیاده‌سازی شده است. از منظر طراحی، این انتخاب هدر سفارش و جزئیات سفارش را جدا می‌کند.

این ساختار چند مزیت دارد:

-هر سفارش می‌تواند هر تعداد آیتم داشته باشد

-امکان محاسبه دقیق مجموع سفارش فراهم است

-امکان گزارش‌گیری از آیتم‌های پرفروش یا سفارش‌های مشتریان وجود دارد

-توسعه قابلیت‌هایی مثل کوپن یا تخفیف در فاز دوم ساده‌تر می‌شود

4-8 .جمع بندی فاز اول

به‌طور خلاصه، فاز اول اسکلت اصلی سامانه را شکل می‌دهد. اگر فاز دوم را حذف کنیم، همچنان پروژه یک سامانه قابل استفاده برای مدیریت کاربران، رستوران، منو و سفارش خواهد بود.

5 .فاز دوم: توسعه منطق تجاری و شخصی‌سازی مشتری

5-1 .هدف فاز دوم

فاز دوم با هدف هوشمندسازی رفتار سیستم نسبت به مشتری طراحی شده است. همانطور که در راهنمای رسمی پروژه آمده است که مشتری بر اساس رفتار و تعداد سفارش‌ها باید در یکی از سطوح عضویت قرار گیرد، امتیاز وفاداری کسب کند و از مزایای متفاوتی مانند تخفیف، هزینه ارسال کمتر و کوپن‌های ماهانه بهره‌مند شود .

5-2 .کلاس انتزاعیMembershipLevel

پایه فاز دوم، کلاس `MembershipLevel` است که به‌صورت abstract طراحی شده است .
`MembershipLevel.h:6`این کلاس یک رابط concrete برای رفتار سطوح مختلف فراهم می‌کند؛ از جمله:

-نام سطح

-درصد تخفیف

-نحوه محاسبه هزینه ارسال

-ضریب دریافت امتیاز

-امتیاز لازم برای سطح بعد

-قابلیت clone برای کپی چندریختی

`MembershipLevel.h:10`

در فاز دوم، برای مدیریت «سطح عضویت» مشتری از یک طراحی ترکیبی شبیه Strategy و State استفاده شده است. ایده اصلی این است که رفتار مشتری بر اساس سطحش عوض شود، بدون اینکه در منطق سفارش، if/else یا switchهای طولانی بنویسیم.

- MembershipLevel نقش رابط /استراتژی را دارد و رفتارهای وابسته به سطح را به‌صورت متدهای abstract تعریف می‌کند: نام سطح، درصد تخفیف، هزینه ارسال، ضریب امتیاز و حدنصاب سطح بعدی. MembershipLevel.h:6-16
- کلاس‌های NormalLevel، SilverLevel، GoldLevel و VipLevel همین interface را override می‌کنند و هر کدام عملیلت های مخصوص خودشان را دارند.
  MembershipLevel.h:19-87
- Customer یک اشاره‌گر به MembershipLevel نگه می‌دارد، نه فقط یک رشته یا عدد ساده. بنابراین رفتار فعلی مشتری از طریق شیء سطحش تعیین می‌شود. -Customer.h:10 13
- در constructor، مشتری با NormalLevel ساخته می‌شود تا همیشه یک سطح معتبر داشته باشد. Customer.h:16-24

- برای کپی‌کردن درست اشیای چندریختی، از clone() استفاده شده تا نوع واقعی level حفظ شود. Customer.h:26-42
- MembershipFactory سطح مناسب را از روی نام یا امتیاز می‌سازد و این ساخت level را از بقیه کد جدا می‌کند. MembershipFactory.h:7-27
- در دیتابیس هم state فعلی با points و level_current ذخیره می‌شود و تاریخچه تغییرات در MembershipHistory ثبت می‌شود. DatabaseManager.cpp:50-58 DatabaseManager.cpp:110-119

از نظر مفهومی:

- از دید Strategy، هر سطح عضویت یک الگوریتم متفاوت برای تخفیف، ارسال و امتیازدهی ارائه می‌دهد.
- از دید State, وقتی سطح مشتری عوض می‌شود، رفتار جاری او هم عوض می‌شود.

یعنی اگر مشتری از Normal به Gold ارتقا پیدا کند، کد اصلی سفارش تغییر نمی‌کند؛ فقط شیء membershipLevel عوض می‌شود و رفتار جدید خودکار اعمال می‌شود که باعث افزایش انعطاف‌پذیری و حذف منطق شرطی پراکنده. MembershipLevel.h:10-16 Customer.h:10-13

5-3 .سطوح عضویت

در `MembershipLevel.h:19` سطوح مختلف عضویت به‌صورت کلاس‌های concrete تعریف شده‌اند:

- `Normal`
- `Silver`

- `Gold`

- `VIP`

هر سطح می‌تواند رفتار متفاوتی در زمینه تخفیف، هزینه ارسال و ضریب امتیاز داشته باشد.

این تصمیم چند مزیت دارد:

-افزودن سطح جدید بدون دست‌زدن به منطق اصلی امکان‌پذیر است

-کد خواناتر می‌شود

-تغییر رفتار یک سطح فقط در همان کلاس انجام می‌شود

-اصل باز-بسته یا OCP بهتر رعایت می‌شود

در فاز دوم، اصل OCP یا Open/Closed Principle این‌طور پیاده‌سازی شده که سیستم برای «افزودن رفتار جدید» باز است، اما برای «تغییر منطق اصلی برنامه» تا حد زیادی بسته نگه داشته شده است.

- هسته این موضوع در کلاس انتزاعی MembershipLevel دیده می‌شود. این کلاس چند عملیات ثابت برای رفتارهای وابسته به سطح تعریف می‌کند: getDiscountPercent، calculateShippingCost، getPointMultiplier و getRequiredPointsForNextLevel. MembershipLevel.h:6-16
- سطح‌های مختلف مثل NormalLevel، SilverLevel، GoldLevel و VipLevel فقط با ارث‌بری از این کلاس و override کردن همین متدها اضافه شده‌اند.
  MembershipLevel.h:19-87
- در نتیجه، منطق مشتری یا سفارش لازم نیست بداند هر level دقیقاً چه فرمولی دارد؛ فقط با interface عمومی MembershipLevel کار می‌کند. MembershipLevel.h:10-16

- Customer هم به‌جای وابستگی به کلاس‌های concrete، فقط یک MembershipLevel* نگه می‌دارد. پس مشتری به abstraction وابسته است، نه به GoldLevel یا VipLevel خاص. Customer.h:10-13
- این وابستگی به abstraction باعث می‌شود تغییر یا توسعه رفتار levelها، کمترین اثر را روی کلاس Customer بگذارد. Customer.h:16-24

5-4 .ارتباط Customer باMembershipLevel

در `Customer.h:13` یک اشاره‌گر یا reference به `MembershipLevel` نگهداری می‌شود. این یعنی هر مشتری در زمان اجرا یک سطح عضویت فعال دارد و رفتار او بر اساس همان شیء تعیین می‌شود.

همچنین در `Customer.h:26` منطق کپی چندریختی سطح عضویت قرار گرفته. این نکته از نظر مدیریت حافظه و جلوگیری از اشتباه در کپی اشیای دارای polymorphism اهمیت دارد.

5-5 MembershipFactory .

برای ساخت سطح عضویت بر اساس نام ذخیره‌شده در دیتابیس، از `MembershipFactory` استفاده شده است `MembershipFactory.h:7` .این کلاس به برنامه کمک می‌کند که هنگام بارگذاری یک مشتری از پایگاه‌داده، سطح مناسب او را بر اساس مقدار `level_current` بسازد.

این الگو وابستگی مستقیم کدهای دیگر به سازنده‌های concrete را کاهش می‌دهد و تغییرپذیری برنامه را بهتر می‌کند.

5-6 .امتیاز وفاداری و ارتقای سطح

بر اساس راهنمای فاز دوم مشتری باید پس از سفارش موفق امتیاز دریافت کند و در صورت عبور از حدنصاب، به‌صورت خودکار ارتقا پیدا کند .

این قابلیت بر دو بخش متکی است:

-نگهداری امتیاز در مدل مشتری و جدول کاربران

-نگهداری منطق سطح در کلاس‌هایmembership

در پایگاه‌داده، ستون `points` در جدول `Users` وجود دارد `DatabaseManager.cpp:57` .
همچنین ستون `level_current` برای نگهداری سطح فعلی ذخیره شده است .
`DatabaseManager.cpp:58`

این برنامه در فاز دوم فقط به منطق حافظه‌ای اکتفا نکرده و داده‌های جدید را به ساختار پایدار سیستم نیز منتقل کرده است.

5-7 downgrade .و تاریخچه تغییر سطح

downgrade در صورت افت فعالیت یا تصمیم مدیر باید امکان‌پذیر باشد که این قابلیت در پنل مدیر سیستم با تعیین حد نصاب غیر فعال بودن و میزان امتیاز کسر شده در صورت غیرفعال شدن امکان پذیر است.

برای ثبت تغییرات سطح، جدول `MembershipHistory` به دیتابیس اضافه شده است .
`DatabaseManager.cpp:110` زیرا فقط نگهداری سطح فعلی برای تحلیل رفتار مشتری کافی نیست. با تاریخچه می‌توان فهمید:

-کاربر چه زمانی ارتقا گرفته است

-آیا افت سطح داشته است

-الگوی وفاداری او در طول زمان چگونه بوده است

5-8 .کوپن تخفیف

در فاز دوم، مفهوم کوپن نیز اضافه شده است. مدل کوپن در `Coupon.h:8` تعریف شده و DAOمربوط به آن در `CouponDAO.h:10` قرار دارد. بر اساس راهنمای فاز دوم سه سطح بالاتر باید کوپن‌های ماهانه هوشمند دریافت کنند تا بتوانند از تخفیف ثابت ۲۰ درصدی روی فاکتور نهایی خود بهره ببرند.

افزودن `Coupon` مزایای زیر را دارد:

-امکان اعمال تخفیف جدا از تخفیف عضویت

-ایجاد تعامل بیشتر میان مدیر رستوران و مشتری

همچنین وجود DAO مستقل برای کوپن به این دلیل است که این قابلیت به‌صورت ساختاری وارد سیستم شده، نه به‌صورت یک کد موقتی در منوها.

5-9 .قابلیت‌های مدیریتی جدید

ادمین باید بتواند آمار کاربران بر اساس level را ببیند، امتیاز یا سطح آن‌ها را تغییر دهد و history تغییر level را مشاهده کند .

این نقش ادمین را نیز توسعه می‌دهد. بنابراین این بخش از فاز دوم دو بعد دارد:

-توسعه رفتار مشتری

-توسعه ابزارهای کنترلی برای مدیریت سیستم

5-10 .جمع بندی فاز دوم

فاز اول اسکلت سیستم را می سازد و فاز دوم منطق تجاری آن را گسترش می دهد. در این مرحله، سامانه از یک سیستم ساده ثبت سفارش به بستری تبدیل شده که بر اساس رفتار مشتری تصمیم‌گیری می‌کند، مزایا اختصاص می‌دهد، وفاداری را اندازه‌گیری می‌کند و داده‌های مرتبط را در تاریخچه ذخیره می‌کند.

6 .خلاصه تحلیلی دو فاز

فاز اول بیشتر بر «قابلیت‌های پایه و زیرساختی» تمرکز دارد:

-تعریف کاربران و نقش‌ها

-مدیریت رستوران و منو

-ثبت و ذخیره سفارش

-طراحی پایگاه‌داده اصلی

-ایجادDAO ها

-ساخت جریان کنسولی برنامه

در حالی که فاز دوم بیشتر بر «هوشمندسازی و توسعه منطق تجاری» متمرکز است:

-امتیاز وفاداری

-سطح عضویت

-ارتقا وdowngrade

-کوپن تخفیف

-تاریخچه تغییرات

-توسعه اختیارات مدیریتی

7 .مفاهیم شی گرایی در پروژه

. Encapsulation7-1

در اکثر کلاس‌ها، داده‌ها به‌صورت private نگهداری می‌شوند و از طریق متدهای دسترسی کنترل می‌شوند. این موضوع در `User.h:7` و `Restaurant.h:11` و `MenuItem.h:37` و `Coupon.h:8` قابل مشاهده است. کپسوله‌سازی باعث می‌شود تغییرات داخلی کلاس‌ها اثر مستقیم روی سایر بخش‌ها نداشته باشد.

Inheritance7-2

‘Customer’,’SystemAdmin’,’RestaurantManager’ از ‘User’ ارث بری می کنند.

این ساختار کمک می‌کند ویژگی‌های مشترک کاربران فقط یک‌بار در کلاس پایه تعریف شود و رفتارهای اختصاصی در کلاس‌های فرزند قرار گیرد.

همچنین در فاز دوم کلاس های ‘NormalLevel’,’SilverLevel’,’GoldLevel’,’VipLevel’ از کلاس پایه ‘MembershipLevel’ ارث بری می کنند.

. Polymorphism7-3

چندریختی در دو بخش پیاده سازی می‌شود:

-وجود destructor مجازی در `User` و قابلیت رفتار چندشکلی در سلسله‌مراتب کاربران . `User.h:11`

-طراحی `MembershipLevel` به‌عنوان کلاس انتزاعی و استفاده ازsubclass های مختلف برای رفتارهای متفاوت عضویت `MembershipLevel.h:6` .

این استفاده از polymorphism به‌خصوص در فاز دوم باعث حذف شرط‌های طولانی و افزایش انعطاف‌پذیری شده است.

. Abstraction7-4

کلاس `MembershipLevel` نمونه بارز abstraction است `MembershipLevel.h:6` .به‌جای اینکه برنامه بداند هر سطح چگونه تخفیف یا هزینه ارسال را حساب می‌کند، فقط با interface عمومی آن کار می‌کند. این موضوع وابستگی را کم و خوانایی طراحی را بیشتر می‌کند.

8 .طراحی پایگاه‌داده

در این پروژه، DatabaseManagerبا الگوی Singletonپیاده‌سازی شده تا کل برنامه فقط یک نمونه از DatabaseManager و در نتیجه یک اتصال مشترک SQLite داشته باشد.

- در DatabaseManager.h یک اشاره‌گر static به نام instance تعریف شده و اتصال دیتابیس هم داخل همان کلاس نگه‌داری می‌شود؛ این یعنی کل برنامه فقط با همان نمونه کار می‌کند. DatabaseManager.h:14

- constructor و destructor خصوصی هستند، پس هیچ کلاسی بیرون از این فایل نمی‌تواند مستقیم DatabaseManager بسازد یا حذف کند. این یکی از شرط‌های اصلی Singleton است. DatabaseManager.h:18
- copy constructor و assignment operator هم حذف شده‌اند تا کسی نتواند از روی نمونه‌ی موجود، کپی جدید بسازد. DatabaseManager.h:21
- ساخت نمونه به‌صورت lazy initialization انجام می‌شود؛ یعنی فقط وقتی اولین بار getInstance() صدا زده شود، شیء ساخته می‌شود. DatabaseManager.cpp:28
- داخل constructor، فایل Foodock.db با sqlite3_open باز می‌شود و اگر اتصال موفق باشد، جدول‌ها هم ساخته می‌شوند. DatabaseManager.cpp:9
- برای بستن اتصال و آزاد کردن حافظه، releaseInstance() در پایان برنامه صدا زده می‌شود و destructor هم sqlite3_close را اجرا می‌کند. DatabaseManager.cpp:35 DatabaseManager.cpp:21
- در main.cpp هم نمونه‌ی Singleton در شروع برنامه گرفته می‌شود و در انتها آزاد می‌شود، پس چرخه‌ی عمر دیتابیس مشخص و متمرکز است. main.cpp:97 main.cpp:134
- DAOها مثل UserDAO، OrderDAO و RestaurantDAO به‌جای ساخت اتصال جدید، از همین نمونه استفاده می‌کنند و با DatabaseManager::getInstance()->getDB() به دیتابیس دسترسی می‌گیرند. UserDAO.cpp:11 OrderDAO.cpp:41 RestaurantDAO.cpp:6

به عبارت دیگر، نقش Singleton در این پروژه این است که:

1. فقط یک اتصال دیتابیس باز بماند.
2. همه‌ی بخش‌ها از همان اتصال مشترک استفاده کنند.

3. مدیریت باز و بسته شدن دیتابیس متمرکز و ساده شود.
4. از ایجاد اتصال‌های تکراری و بی‌نظم جلوگیری شود.

در فاز اول، جدول‌های اصلیUsers ، Restaurants، MenuItems، Ordersو OrderItems ساخته شده‌اند `DatabaseManager.cpp:62` `DatabaseManager.cpp:51` . `DatabaseManager.cpp:87` `DatabaseManager.cpp:74` `DatabaseManager.cpp:99`

در فاز دوم، جدول‌ها و ستون‌های جدیدی برای پشتیبانی از loyalty اضافه شده‌اند:

-ستون `points` در `DatabaseManager.cpp:57` . `Users`

-ستون `level_current` در `DatabaseManager.cpp:58` . `Users`

-جدول `DatabaseManager.cpp:110` . `MembershipHistory`

همچنین برخی ویژگی‌های تکمیلی مانند `prep_time` برای رستوران و `is_special` برای آیتم منو در پایگاه داده قرار گرفته است `DatabaseManager.cpp:83` `DatabaseManager.cpp:67` .

همچنین ستون‌های 'prepTime`,`weight` و `volume` برای `MenuItems` در پایگاه داده قرار گرفته است تا به مدیر رستوران دسترسی های بیشتری برای شخصی سازی منو داده شود.

9 .مدیریت خطا و حافظه

1. مدیریت خطا در فاز 1 و فاز 2

در کل پروژه، خطاها با این روش‌ها کنترل شده‌اند:

- بررسی نتیجه‌ی توابع SQLite

- برگرداندن true/false
- چک کردن NULL
- چاپ پیام خطا روی cout
- پاک‌سازی بعد از خطا با sqlite3_finalize() یا sqlite3_free()
- اعتبارسنجی ورودی کاربر و درخواست دوباره در صورت خطا

.1 خطاهای دیتابیس و SQLite

1. خطای باز شدن دیتابیس

در DatabaseManager، باز شدن فایل دیتابیس بررسی می‌شود:

- sqlite3_open()
- اگر موفق نباشد، sqlite3_errmsg(db) چاپ می‌شود
- سپس db = NULL می‌شود

تابع مسئول:

- DatabaseManager::DatabaseManager()
- DatabaseManager.cpp:9-18

2. خطای مقداردهی اولیه دیتابیس در شروع برنامه

بعد از getInstance()، برنامه چک می‌کند که نمونه ساخته شده یا نه:

- اگر db == nullptr باشد، برنامه با پیام خطا متوقف می‌شود

تابع مسئول:

- بخش شروع برنامه در main.cpp
- main.cpp:97-101

3. خطا در اجرای SQL

در DAOها، این الگو ها باعث می‌شود:

- sqlite3_prepare_v2()
- sqlite3_step()
- بررسی SQLITE_OKیا SQLITE_DONE
- اگر خطا رخ بدهد، تابع falseبرمی‌گرداند

توابع مسئول:

- UserDAO
- OrderDAO
- RestaurantDAO
- MenuItemDAO
- CouponDAO
- OrderItemDAO

نمونه‌ها:

- UserDAO.cpp:18-44
- RestaurantDAO.cpp:18-72

- OrderItemDAO.cpp:80-147
- OrderDAO.cpp:40-58

4. پاک‌کردن خطای SQLite

در بعضی عملیات مثل RestaurantDAO، اگر sqlite3_exec() شکست بخورد، رشته‌ی خطا با sqlite3_free(errMsg) آزاد می‌شود.

این خیلی مهم است چون اگر آزاد نشود، **memory leak** رخ می‌دهد.

تابع مسئول:

- RestaurantDAOدر عملیاتadd/update/delete
- RestaurantDAO.cpp:18-72

.3خطاهای مربوط به ورودی کاربر

در main.cpp چند تابع برای اعتبارسنجی ورودی وجود دارد که در عمل نقش مدیریت خطا را هم دارند:

.1تابع readNonEmotyString

اگر رشته خالی یا فقط شامل فاصله باشد، آن را رد می‌کند.

.2تابع readPositiveInt و readPositiveDouble

رشته خالی و عدد منفی را رد می‌کنند و دوباره درخواست ورودی می‌دهند.

تابع مسئول اصلی:

- توابع کمکی ورودی در main.cpp

- main.cpp:143-229

3. مدیریت خطا در فاز 2

در فاز 2 علاوه بر خطاهای دیتابیس، خطاهای مربوط به چندریختی هم اضافه شده‌اند:

1. بررسی null بودن مشتری

در OrderDAO اگر مشتری پیدا نشود:

- تابع ادامه نمی‌دهد
- NULL را بررسی می‌کند و از کرش جلوگیری می‌شود

تابع مسئول:

- OrderDAO هنگام پردازش سفارش
- OrderDAO.cpp:8-36

2. خطا در آماده‌سازی membership history

در ثبت تاریخچه عضویت هم اگر statement آماده نشود، پیام خطا چاپ می‌شود و تابع false برمی‌گرداند.

تابع مسئول:

- OrderDAO::insertMembershipHistory()
- OrderDAO.cpp:40-58

2. مدیریت حافظه در فاز 1 و فاز 2

مدیریت حافظه در پروژه به دو بخش اصلی تقسیم می‌شود:

1. **حافظه‌ی مربوط به دیتابیس و Singleton**
2. **حافظه‌ی مربوط به اشیای پویا، مخصوصاً در فاز 2**

.1مدیریت حافظه در فاز 1

1. Singleton دیتابیس

DatabaseManager به‌صورت پویا ساخته می‌شود:

- در getInstance() با new ساخته می‌شود
- در releaseInstance() با delete آزاد می‌شود

توابع مسئول:

- DatabaseManager::getInstance()
- DatabaseManager::releaseInstance()
- destructor
- DatabaseManager.cpp:28-40

2. بستن اتصال دیتابیس

در destructor، اتصال SQLite با sqlite3_close(db) بسته می‌شود.

این از ماندن connection باز جلوگیری می‌کند.

تابع مسئول:

- DatabaseManager::~DatabaseManager()

- DatabaseManager.cpp:21-26

3. آزادسازی statementهای SQLite

در DAOها بعد از هر query یا execution، این تابع صدا زده می‌شود:

- sqlite3_finalize(stmt)

این برای جلوگیری از نشت حافظه‌ی statementها لازم است.

توابع مسئول:

- UserDAO
- OrderItemDAO
- OrderDAO
- MenuItemDAO
- CouponDAO

نمونه‌ها:

- UserDAO.cpp:18-64
- OrderItemDAO.cpp:80-147

.2مدیریت حافظه در فاز 2

1. حافظه‌ی MembershipLevel

در کلاس Customer یک اشاره‌گر به MembershipLevel نگه‌داری می‌شود:

- MembershipLevel* membershipLevel

در constructor، این اشاره‌گر با new NormalLevel() مقداردهی می‌شود.

تابع مسئول:

- constructor Customer
- Customer.h:10-24

2. deep copy با clone()

برای اینکه کپی اشیای مشتری درست انجام شود، از clone() استفاده شده:

- این باعث می‌شود فقط pointer کپی نشود
- بلکه شیء واقعی از نوع درست ساخته شود

توابع مسئول:

- MembershipLevel::clone()
- copy constructorدر Customer
- assignment operatorدر Customer
- MembershipLevel.h:8-16
- Customer.h:26-44

3. جلوگیری از memory leak در assignment

در operator=، قبل از جایگزین‌کردن level جدید:

- delete membershipLevel;

این کار مهم است چون اگر سطح قبلی حذف نشود، حافظه نشت می‌کند.

تابع مسئول:

- Customer::operator=
- Customer.h:36-44

4. حذف مشتری پویا بعد از استفاده

در OrderDAO، مشتری با getCustomerById() به‌صورت پویا گرفته می‌شود:

- بعد از استفاده، باید delete customer; انجام شود

اگر این کار انجام نشود، memory leak رخ می‌دهد.

تابع مسئول:

- OrderDAO
- OrderDAO.cpp:8-36

5. destructor مجازی

در MembershipLevel destructor مجازی تعریف شده:

- virtual ~MembershipLevel() {}

این باعث می‌شود اگر یک شیء فرزند از طریق pointer پایه حذف شود، تخریب درست انجام شود.

تابع مسئول:

- MembershipLevel::~MembershipLevel()
- MembershipLevel.h:8-16

10 . روابط اصلی و class diagram

Class diagram این برنامه با استفاده از نرم افزار StarUML رسم شده است.

`User` -به‌عنوان کلاس پایه

`Customer` -، `RestaurantManager` و `SystemAdmin` به‌عنوان زیرکلاس‌های `User`

`Customer` -دارای رابطه با `MembershipLevel`

`MembershipLevel` -به‌عنوان کلاس انتزاعی

`Normal` -، `Gold`, `Silver` و `VIP` به‌عنوان زیرکلاس‌های `MembershipLevel`

`Restaurant` -دارای چند `MenuItem`

`Order` -دارای چند `OrderItem`

`Order` -مرتبط با `Customer`

`Order` -مرتبط با `Restaurant`

`Coupon` -مرتبط با منطق مشتری یاDAO های کوپن

یا به عبارت دیگر :

Inheritance:

- Customer -> User

- RestaurantManager -> User

- SystemAdmin -> User

- NormalLevel -> MembershipLevel

- SilverLevel -> MembershipLevel

- GoldLevel -> MembershipLevel

- VipLevel -> MembershipLevel

:Composition

- Customer 1 *-- 1 MembershipLevel

- Order 1 *-- 1..* OrderItem

:Associations

- RestaurantManager 1 -- 0..1 Restaurant : manages

- Restaurant 1 -- 0..* MenuItem : offers

- Customer 1 -- 0..* Order : places

- Restaurant 1 -- 0..* Order : receives

- MenuItem 1 -- 0..* OrderItem : referencedBy

- User 1 -- 0..* Coupon : owns

:Dependencies

- Customer ..> MembershipFactory

MembershipFactory ..> MembershipLevel -

MembershipFactory ..> NormalLevel -

MembershipFactory ..> SilverLevel -

MembershipFactory ..> GoldLevel -

MembershipFactory ..> VipLevel -

UserDAO ..> User -

UserDAO ..> Customer -

RestaurantDAO ..> Restaurant -

MenuItemDAO ..> MenuItem -

OrderDAO ..> Order -

OrderItemDAO ..> OrderItem -

CouponDAO ..> Coupon -

UserDAO ..> DatabaseManager -

RestaurantDAO ..> DatabaseManager -

MenuItemDAO ..> DatabaseManager -

OrderDAO ..> DatabaseManager -

OrderItemDAO ..> DatabaseManager -

CouponDAO ..> DatabaseManager -

DatabaseManager ..> sqlite3 -

11 .سناریوهای تست

-سناریوی 1: ورود به عنوان ادمین و افزودن کاربر و حذف رستوران و مشاهده گزارش های فروش و سفارش ها و کاربران فعال و سطوح کاربران

۱.ورود به عنوان ادمین:

==========

Welcome to Foodock

==========

Login .1

Register as Manager .2

Register as Customer .3

Exit .0

Enter your choice: 1

Enter username: admin

Enter password: 1234

۲.افزودن کاربر:

===== Admin Panel =====

Welcome, System Admin

Show All Users .1

Show All Restaurants .2

Add User .3

Delete User .4

Add Restaurant .5

Update Restaurant .6

Delete Restaurant .7

Modify User Points/Level .8

Apply Inactivity Downgrade .9

View Level Change Logs .10

Reports .11

Logout .0

Enter your choice: 3

===== Add User =====

Add User .1

Back .0

Enter your choice: 1

Enter name: Customer 4

Enter username: &customer

Enter password: 1234

Enter role (customer/manager/admin): customer

User added successfully

۳.بازگشت به پنل ادمین و حذف رستوران:

===== Add User =====

Add User .1

Back .0

Enter your choice: 0

===== Admin Panel =====

Welcome, System Admin

Show All Users .1

Show All Restaurants .2

Add User .3

Delete User .4

Add Restaurant .5

Update Restaurant .6

Delete Restaurant .7

Modify User Points/Level .8

Apply Inactivity Downgrade .9

View Level Change Logs .10

Reports .11

Logout .0

Enter your choice: 7

===== Delete Restaurant =====

Show All Restaurants .1

Delete Restaurant .2

Back .0

Enter your choice: 2

===== Restaurants =====

ID: 1 | Name: Foodock | Address: Main Street | Status: active | PrepTime: 25 | Contact: 09130000000 | Manager ID: 2

ID: 2 | Name: Hometown | Address: Main Street | Status: active | PrepTime: 25 | Contact: 09150000000 | Manager ID: 5

Enter restaurant ID to delete: 2

Restaurant deleted successfully

۴.بازگشت به پنل ادمین و ورود به بخش گزارش ها و انتخاب گزارش:

===== Delete Restaurant =====

Show All Restaurants .1

Delete Restaurant .2

Back .0

Enter your choice: 0

===== Admin Panel =====

Welcome, System Admin

Show All Users .1

Show All Restaurants .2

Add User .3

Delete User .4

Add Restaurant .5

Update Restaurant .6

Delete Restaurant .7

Modify User Points/Level .8

Apply Inactivity Downgrade .9

View Level Change Logs .10

Reports .11

Logout .0

Enter your choice: 11

===== Users Reports =====

Total Sales .1

Total Orders .2

Active Users .3

User Membership Level Statistics .4

Back .0

Enter your choice: 1

===== Total Sales Report =====

Total Sales: 160.045

===== Users Reports =====

Total Sales .1

Total Orders .2

Active Users .3

User Membership Level Statistics .4

Back .0

Enter your choice: 2

===== Total Orders Report =====

Total Orders: 7

===== Users Reports =====

Total Sales .1

Total Orders .2

Active Users .3

User Membership Level Statistics .4

Back .0

Enter your choice: 3

===== Active Users Report =====

.Active users are customers who have placed at least one order

User ID: 3 | Name: Customer One | Username: customer | Role: customer | Orders: 6

User ID: 6 | Name: Customer 3 | Username: #customer | Role: customer | Orders: 1

Total Active Users: 2

===== Users Reports =====

Total Sales .1

Total Orders .2

Active Users .3

User Membership Level Statistics .4

Back .0

Enter your choice: 4

===== User Count by Membership Level =====

VIP: 1 users -

Gold: 0 users -

Silver: 0 users -

Normal: 3 users -

==========

-سناریوی 2: ورود به عنوان مدیر رستوران و مشاهده منوی رستوران و افزودن آیتم منو توسط مدیر رستوران و مشاهده دوباره منو و بروزرسانی اطلاعات رستوران و مشاهده سفارش ها و بروزرسانی وضعیت یک سفارش

۱.ورود به عنوان مدیر رستوران:

==========

Welcome to Foodock

==========

Login .1

Register as Manager .2

Register as Customer .3

Exit .0

Enter your choice: 1

Enter username: manager

Enter password: 1234

۲.مشاهده منو ,افزودن آیتم به منو و مشاهده دوباره منو:

===== Manager Panel =====

Restaurant: Foodock

View My Menu Items .1

Add Menu Item .2

Update Menu Item .3

Delete Menu Item .4

View Orders .5

Update Restaurant Info .6

Update Order Status .7

Logout .0

Enter your choice: 1

===== Menu Items =====

ID: 1 | Name: Burger | Category: Main Course | Price: 10.5 | Available: yes | Description: ... | Special: no

ID: 2 | Name: Lemonade | Category: drink | Price: 5.5 | Available: yes | Description: ... | Special: no

...Press Enter to continue

===== Manager Panel =====

Restaurant: Foodock

View My Menu Items .1

Add Menu Item .2

Update Menu Item .3

Delete Menu Item .4

View Orders .5

Update Restaurant Info .6

Update Order Status .7

Logout .0

Enter your choice: 2

===== Add Menu Item =====

Add Menu Item .1

Back .0

Enter your choice: 1

Enter item name: Water

:Select category

Appetizer .1

Main Course .2

Dessert .3

Drink .4

Enter your choice (1-4): 4

Enter preparation time: 1

Enter volume: 250

Enter price: 3

Enter available (1/0): 1

Enter description: Room Temperature

Enter is special (1/0): 0

Menu item added successfully

...Press Enter to continue

===== Add Menu Item =====

Add Menu Item .1

Back .0

Enter your choice: 0

===== Manager Panel =====

Restaurant: Foodock

View My Menu Items .1

Add Menu Item .2

Update Menu Item .3

Delete Menu Item .4

View Orders .5

Update Restaurant Info .6

Update Order Status .7

Logout .0

Enter your choice: 1

===== Menu Items =====

ID: 1 | Name: Burger | Category: Main Course | Price: 10.5 | Available: yes | Description: ... | Special: no

ID: 2 | Name: Lemonade | Category: drink | Price: 5.5 | Available: yes | Description: ... | Special: no

ID: 3 | Name: Water | Category: drink | Price: 3 | Available: yes | Description: Room Temperature | Special: no

-سناریوی 3: مشاهده همه کاربران,حذف چند کاربر توسط ادمین سیستم و مشاهده دوباره همه کاربران

۱.ورود به عنوان ادمین:

==========

Welcome to Foodock

==========

Login .1

Register as Manager .2

Register as Customer .3

Exit .0

Enter your choice: 1

Enter username: admin

Enter password: 1234

۲.انتخاب منوی حذف کاربر و مشاهده همه کاربران و حذف یک مشتری و یک مدیر رستوران:

===== Admin Panel =====

Welcome, System Admin

Show All Users .1

Show All Restaurants .2

Add User .3

Delete User .4

Add Restaurant .5

Update Restaurant .6

Delete Restaurant .7

Modify User Points/Level .8

Apply Inactivity Downgrade .9

View Level Change Logs .10

Reports .11

Logout .0

Enter your choice: 4

===== Delete User =====

Show All Users .1

Delete User .2

Back .0

Enter your choice: 1

===== Users =====

ID: 1 | Name: System Admin | Username: admin | Role: admin

ID: 2 | Name: Manager One | Username: manager | Role: manager

ID: 3 | Name: Customer One | Username: customer | Role: customer

ID: 4 | Name: Customer 2 | Username: @customer | Role: customer

ID: 5 | Name: Manager 2 | Username: @manager | Role: manager

ID: 6 | Name: Customer 3 | Username: #customer | Role: customer

ID: 7 | Name: Customer 3 | Username: &customer | Role: customer

...Press Enter to continue

===== Delete User =====

Show All Users .1

Delete User .2

Back .0

Enter your choice: 2

Enter user ID to delete: 7

User deleted successfully

===== Delete User =====

Show All Users .1

Delete User .2

Back .0

Enter your choice: 2

===== Delete User =====

Show All Users .1

Delete User .2

Back .0

Enter your choice: 2

Enter user ID to delete: 5

User deleted successfully

===== Delete User =====

Show All Users .1

Delete User .2

Back .0

Enter your choice: 1

===== Users =====

ID: 1 | Name: System Admin | Username: admin | Role: admin

ID: 2 | Name: Manager One | Username: manager | Role: manager

ID: 3 | Name: Customer One | Username: customer | Role: customer

ID: 4 | Name: Customer 2 | Username: @customer | Role: customer

ID: 6 | Name: Customer 3 | Username: #customer | Role: customer

-سناریوی 4: ثبت سفارش و ارتقای خودکار سطح مشتری:

۱.ثبت نام به عنوان مشتری جدید:

Enter your choice: 3

Enter name: Customer 3

Enter username: #customer

Enter password: 1234

۲.ورود به حساب به عنوان مشتری:

Enter your choice: 1

Enter username: #customer

Enter password: 1234

۳.انتخاب سبد خرید و افزودن آیتم ها به سبد:

===== Membership Status =====

Level: Normal

Points: 0

Active Coupons: 0

Points needed for next level: 100

==========

===== Customer Panel =====

Welcome, Customer 3

View Restaurants .1

View Restaurant Menu .2

Shopping Cart .3

Finalize Order .4

View My Orders .5

Logout .0

Enter your choice: 3

===== Shopping Cart =====

.Cart is empty

Preview Earned Points: 0

:Shopping Cart Menu

Add item to cart .1

Decrease/remove item from cart .2

Clear cart .3

Back to customer panel .0

Enter your choice: 1

===== Restaurants =====

ID: 1 | Name: Foodock | Address: Main Street | Status: active | PrepTime: 25 | Contact: 09130000000 | Manager ID: 2

ID: 2 | Name: Hometown | Address: Main Street | Status: active | PrepTime: 25 | Contact: 09150000000 | Manager ID: 5

Enter restaurant ID: 1

===== Available Items =====

ID: 1 | Name: Burger | Price: 10.5

ID: 2 | Name: Lemonade | Price: 5.5

Enter item ID to add: 1

Enter quantity: 5

.Item added to cart successfully

Added Item Preview Points: 52

Current Cart Total Price: 52.5

Preview Earned Points: 52

۴.بازگشت به پنل مشتری و نهایی کردن سفارش:

:Shopping Cart Menu

Add item to cart .1

Decrease/remove item from cart .2

Clear cart .3

Back to customer panel .0

Enter your choice: 0

===== Customer Panel =====

Welcome, Customer 3

View Restaurants .1

View Restaurant Menu .2

Shopping Cart .3

Finalize Order .4

View My Orders .5

Logout .0

Enter your choice: 4

===== Final Order Preview =====

Item: Burger | Quantity: 5 | Unit Price: 10.5 | Subtotal: 52.5

Base Price: 52.5

Membership Discount Amount: 0

Coupon Discount Amount: 0

Shipping Cost: 50

Total Price: 102.5

Estimated Earned Points: 102

Current Loyalty Points: 0

Membership Level: Normal

Finalize this order? (1 = Yes, 0 = No): 1

.Order finalized successfully

Order ID: 6

Final Paid Amount: 102.5

Points Earned: 102

.Cart is now empty

۵.خروج و ورود دوباره برای بروزرسانی سطح:

===== Customer Panel =====

Welcome, Customer 3

View Restaurants .1

View Restaurant Menu .2

Shopping Cart .3

Finalize Order .4

View My Orders .5

Logout .0

Enter your choice: 0

...Logging out

==========

Welcome to Foodock

==========

Login .1

Register as Manager .2

Register as Customer .3

Exit .0

Enter your choice: 1

Enter username: #customer

Enter password: 1234

===== Membership Status =====

Level: Silver

Points: 102

Active Coupons: 0

Points needed for next level: 198

==========

===== Customer Panel =====

Welcome, Customer 3

View Restaurants .1

View Restaurant Menu .2

Shopping Cart .3

Finalize Order .4

View My Orders .5

Logout .0

:Enter your choice

مشاهده می شود که با ثبت یک سفارش بالای ۱۰۰ امتیاز سطح کاربر از normal به silver ارتقا یافت.

-سناریوی 5: اعمال تخفیف و هزینه ارسال رایگان برای کاربر VIP

۱.ورود به عنوان کاربر Vip

==========

Welcome to Foodock

==========

Login .1

Register as Manager .2

Register as Customer .3

Exit .0

Enter your choice: 1

Enter username: customer

Enter password: 1234

===== Membership Status =====

Level: VIP

Points: 797

Active Coupons: 0

.You are at the highest level

==========

۲. انتخاب سبد خرید و انتخاب رستوران و انتخاب آیتم و ثبت سفارش:

===== Customer Panel =====

Welcome, Customer One

View Restaurants .1

View Restaurant Menu .2

Shopping Cart .3

Finalize Order .4

View My Orders .5

Logout .0

Enter your choice: 3

===== Shopping Cart =====

.Cart is empty

Preview Earned Points: 0

:Shopping Cart Menu

Add item to cart .1

Decrease/remove item from cart .2

Clear cart .3

Back to customer panel .0

Enter your choice: 1

===== Restaurants =====

ID: 1 | Name: Foodock | Address: Main Street | Status: active | PrepTime: 25 | Contact: 09130000000 | Manager ID: 2

ID: 2 | Name: Hometown | Address: Main Street | Status: active | PrepTime: 25 | Contact: 09150000000 | Manager ID: 5

Enter restaurant ID: 1

===== Available Items =====

ID: 1 | Name: Burger | Price: 10.5

ID: 2 | Name: Lemonade | Price: 5.5

Enter item ID to add: 1

Enter quantity: 2

.Item added to cart successfully

Added Item Preview Points: 42

Current Cart Total Price: 21

Preview Earned Points: 42

...Press Enter to continue

===== Shopping Cart =====

Restaurant ID: 1

:Items

Item ID: 1 | Name: Burger | Quantity: 2 | Unit Price: 10.5 | Subtotal: 21 .1

Cart Total Price: 21

Preview Earned Points: 42

۳.بازگشت به پنل مشتری و نهایی کردن سفارش:

:Shopping Cart Menu

Add item to cart .1

Decrease/remove item from cart .2

Clear cart .3

Back to customer panel .0

Enter your choice: 0

===== Customer Panel =====

Welcome, Customer One

View Restaurants .1

View Restaurant Menu .2

Shopping Cart .3

Finalize Order .4

View My Orders .5

Logout .0

Enter your choice: 4

===== Final Order Preview =====

Item: Burger | Quantity: 2 | Unit Price: 10.5 | Subtotal: 21

Base Price: 21

Membership Discount Amount: 3.15

Coupon Discount Amount: 0

Shipping Cost: 0

Total Price: 17.85

Estimated Earned Points: 35

Current Loyalty Points: 797

Membership Level: VIP

Finalize this order? (1 = Yes, 0 = No): 1

.Order finalized successfully

Order ID: 7

Final Paid Amount: 17.85

Points Earned: 35

.Cart is now empty

-سناریوی 6: تنزل سطح مشتری توسط ادمین و ثبت تاریخچه

۱. ورود به عنوان ادمین:

(ادمین به دلیل رعایت امنیت برنامه به صورت پیش فرض تعریف می شود.)

```
==================================

Welcome to Foodock

==================================

Login .1

Register as Manager .2

Register as Customer .3

Exit .0

--------------------------------

Enter your choice: 1

Enter username: admin

Enter password: 1234
```

۲.ورود به منوی تغییر امتیاز و انتخاب مشتری و اعمال تغییر:

```
===== Admin Panel =====

Welcome, System Admin

Show All Users .1

Show All Restaurants .2
```

Add User .3

Delete User .4

Add Restaurant .5

Update Restaurant .6

Delete Restaurant .7

Modify User Points/Level .8

Apply Inactivity Downgrade .9

View Level Change Logs .10

Reports .11

Logout .0

Enter your choice: 8

===== Modify Customer Loyalty (Admin) =====

Enter Customer User ID: 6

:Current details for Customer 3

Current Points: 102 -

Current Level: Silver -

Enter new loyalty points: 99

.[Success] Loyalty points updated in database

.[Info] Level changed from Silver to Normal

۳.مشاهده لاگ تغییرات سطح:

```
===== Admin Panel =====
 Welcome, System Admin
1. Show All Users
2. Show All Restaurants
3. Add User
4. Delete User
5. Add Restaurant
6. Update Restaurant
7. Delete Restaurant
8. Modify User Points/Level
9. Apply Inactivity Downgrade
10. View Level Change Logs
11. Reports
0. Logout
 Enter your choice: 10

===== Membership Level Change Logs =====
LogID | UserID   | Customer Name   | Old Level    | New Level    | Points   | Changed At
-------------------------------------------------------------------------------------
3     | 6        | Customer 3      | Silver       | Normal       | 99       | 2026-07-17 15:58:24
2     | 6        | Customer 3      | Normal       | Silver       | 102      | 2026-07-17 15:35:48
1     | 3        | Customer One    | Normal       | VIP          | 700      | 2026-07-14 07:35:22
```

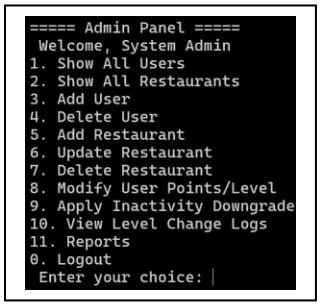
12 .تصاویر و مستندات اجرایی

-تصویر منوی اصلی برنامه:

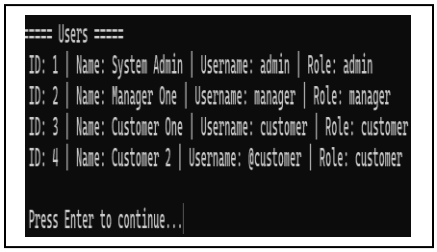
```
Database opened successfully.

==================================
          Welcome to Foodock
==================================
1. Login
2. Register as Manager
3. Register as Customer
0. Exit
----------------------------------
Enter your choice: 1
Enter username: admin
Enter password: 1234
```

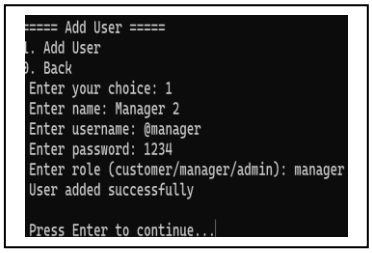
-تصویر پنل ادمین:



نمایش تمام کاربران:



اضافه کردن کاربران:



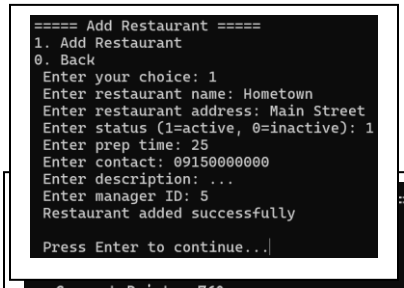
حذف کلربران:



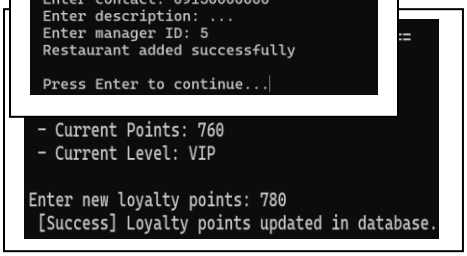
نمایش تمام رستوران ها:



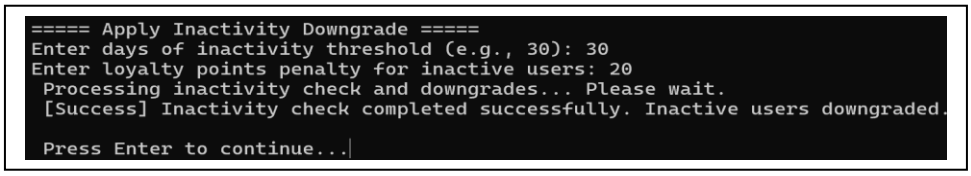
اضافه کردن رستوران:



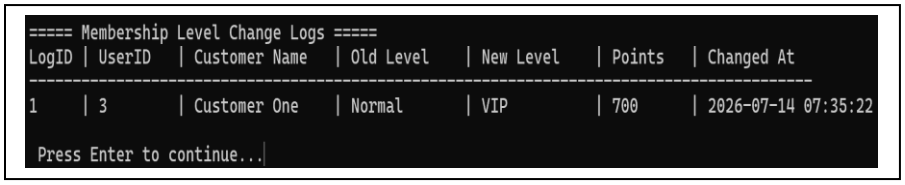
تغییر دستی سطح و امتیاز مشتری:



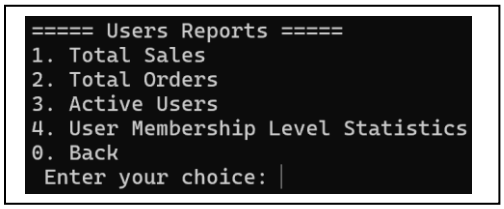
کاهش امتیاز خودکار به دلیل کاهش فعالیت:

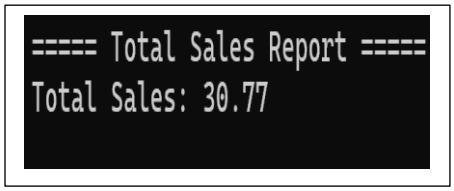


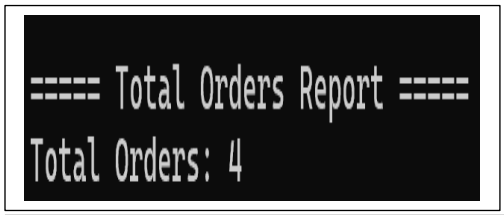
لاگ تغییرات سطح مشتریان:

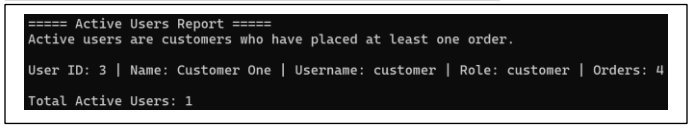


گزارش ها در پنل ادمین:



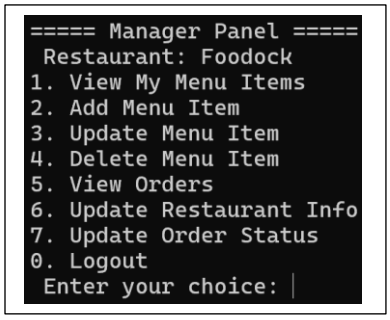








تصویر پنل مدیر رستوران:



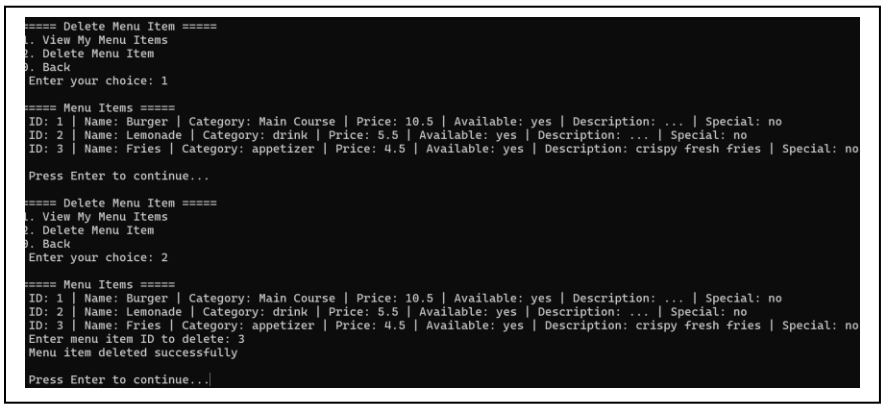
نمایش منوی رستوران مدیر:



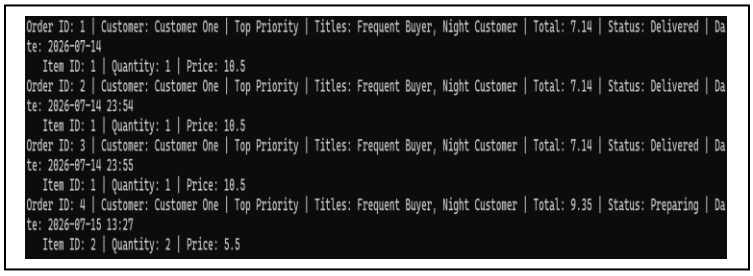
اضافه کردن آیتم به منو:



حذف آیتم از منو:



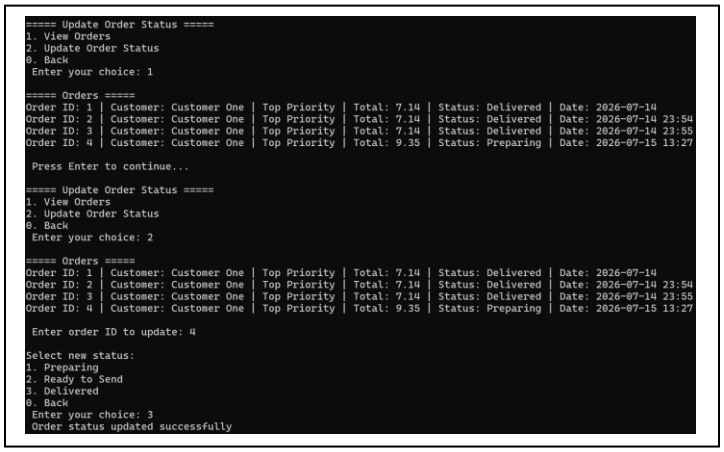
نمایش تمامی سفارش های رستوران:



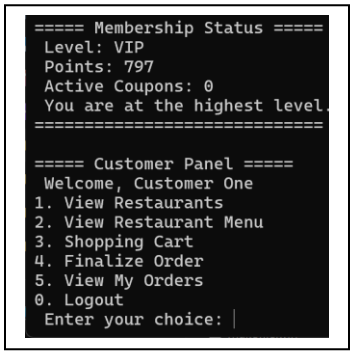
بروزرسانی اطلاعات رسنوران:



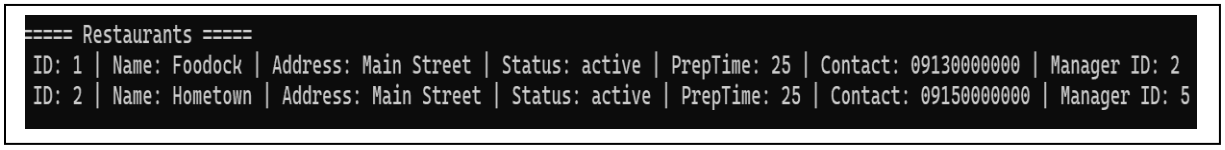
بروزرسانی وضعیت سفارش ها:



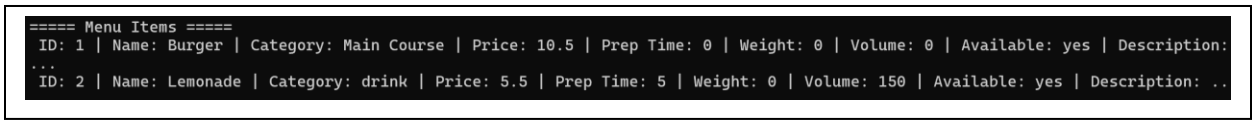
-تصویر پنل مشتری:



نمایش همه رستوران ها به مشتری:



نمایش آینم های منو به مشتری:

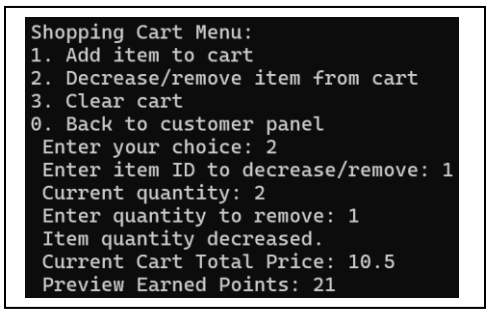


سبد خرید مشتری:

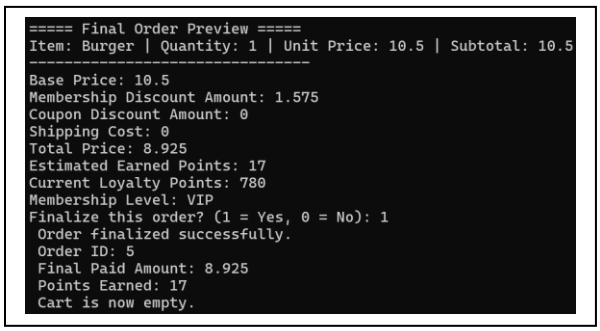
اضافه کردن به سبد خرید:



حذف آیتم از سبد خرید:



نهایی کردن سفارش:



مشاهده سوابق سفارش:



13 .جمع بندی نهایی طراحی

-استفاده از `DatabaseManager` به‌صورت Singleton برای مدیریت اتصال مشترک .
`DatabaseManager.h:10`

-استفاده از DAO برای جداسازی منطق SQL از رابط کاربر `UserDAO.h:9` .

-استفاده مناسب از وراثت در مدل کاربران `Customer.h:10` .

-استفاده قوی از polymorphism در طراحی `MembershipLevel.h:6` .membership

-امکان توسعه‌پذیری بالا برای افزودن سطح جدید یا قابلیت‌های جدید

14 .جمع‌بندی کلی

این پروژه در دو فاز توسعه یافته و هر دو فاز نقش اساسی در کامل شدن سامانه دارند. فاز اول زیرساخت اصلی را می‌سازد: مدیریت کاربران، نقش‌ها، رستوران‌ها، منو، سفارش‌ها، طراحی پایگاه‌داده و لایه `OrderDAO.h:1` `DatabaseManager.cpp:51` `main.cpp:96` DAO.

فاز دوم این زیرساخت را ارتقا می‌دهد و با افزودن سطح عضویت، امتیاز وفاداری، کوپن، تاریخچه سطح و رفتارهای مدیریتی جدید، سامانه را هوشمندتر و واقعی‌تر می‌کند .
`Coupon.h:8` `MembershipFactory.h:7` `MembershipLevel.h:6`
`DatabaseManager.cpp:110`

15مشاهده مخزن Github

https://github.com/mahyarmohajer37-hub/Foodock